

Algorithmes et Pensée Computationnelle

Algorithmes spatiaux

Le but de cette séance est de comprendre les caractéristiques des données spatiales et les structures de données couramment utilisées pour les manipuler. Lors de cette séance, nous implémenterons des algorithmes de manipulation de structures de données spatiales présentés en cours. Au terme de la séance, l'étudiant sera en mesure d'utiliser plusieurs algorithmes spatiaux pour résoudre des problèmes de base de façon efficiente.

1 Nearest-Neighbor

Dans cette section, nous allons implémenter une recherche du plus proche voisin. Le but de cette méthode est de trouver le voisin le plus proche du point de départ en tenant compte de ce point de "départ" et un ensemble de points. Nous allons implémenter cet algorithme et ensuite l'étendre à un algorithme des "k plus proches voisins" (recherche des k voisins les plus proches plutôt que du seul voisin le plus proche). Nous vous recommandons de traiter les questions dans l'ordre.

Question 1: (🕒 5 minutes) La fonction de distance : Python

Pour implémenter notre recherche, nous avons besoin d'écrire une fonction permettant de calculer la distance entre 2 points. Ecrivez une fonction qui permet de calculer la distance entre 2 points.

💡 Conseil

Soit 2 points en 2 dimensions (x_1, y_1) et (x_2, y_2) , la distance euclidienne entre ces 2 points est donnée par : $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. En Python, la fonction permettant de faire une racine carrée est `sqrt` de la librairie `math`. Pour mettre au carré, on utilise `nombre**2`.

Question 2: (🕒 10 minutes) Nearest-neighbor search

Implémentez la recherche du voisin le plus proche. Cette dernière fonctionne de la façon suivante :

1. Traversez chaque point.
2. Pour chaque point, calculez la distance entre ce point et le point de départ.
3. Retournez un tuple contenant les coordonnées du point le plus proche sous forme d'une liste et la distance.

💡 Conseil

Utilisez la fonction de distance de la Question 1 et parcourez les points à l'aide d'une boucle `for`. Si votre input est `[(2, 3), (5, 6), (1, 4), (2, 4), (3, 5)]` et que le point de départ est `[4, 4]`, alors l'output devra être `([3, 5] 1.414)`. Ici, `[3, 5]` sont les coordonnées du point le plus proche et `1.414` est la distance euclidienne entre notre point de départ et le point le plus proche.

Note : Pour que votre programme fonctionne, écrivez votre algorithme du plus proche voisin dans le même programme que celui de la Question 1.

Question 3: (🕒 15 minutes) K-nearest-neighbor search

Améliorez l'algorithme du voisin le plus proche effectué plus haut afin qu'il puisse retourner des *K*-plus proches voisins.

💡 Conseil

Appliquez l'algorithme du plus proche voisin K -fois. À la fin de chaque itération, retirez le voisin le plus proche de l'ensemble des points sur lequel l'algorithme s'applique. De cette façon, vous trouverez le second voisin le plus proche, le troisième, etc...

Votre fonction devra retourner une liste sous la forme : $[(x_1, y_1, distance1), (x_2, y_2, distance2), \dots]$. En considérant un input $[(2, 3), (5, 6), (1, 4), (2, 4), (3, 5)]$, un point de départ $(4, 4)$ et un nombre de voisins $K = 2$, l'output de votre algorithme devra être : $[(3, 5, 1.4142135623730951), (2, 4, 2.0)]$. Attention au type des variables que vous manipulez. Pour rappel, les tuples sont immuables en Python. Pensez à créer de nouveaux tuples contenant les coordonnées des points et leurs distances.

Note : Pour que votre programme fonctionne, écrivez votre algorithme dans le même programme que celui de la Question 1 et la Question 2.

2 K-dimensional tree

Question 4: (🕒 5 minutes) Lecture du code — Construction d'un KD-Tree

Algorithm 1 BuildKDTree(points, depth, k)

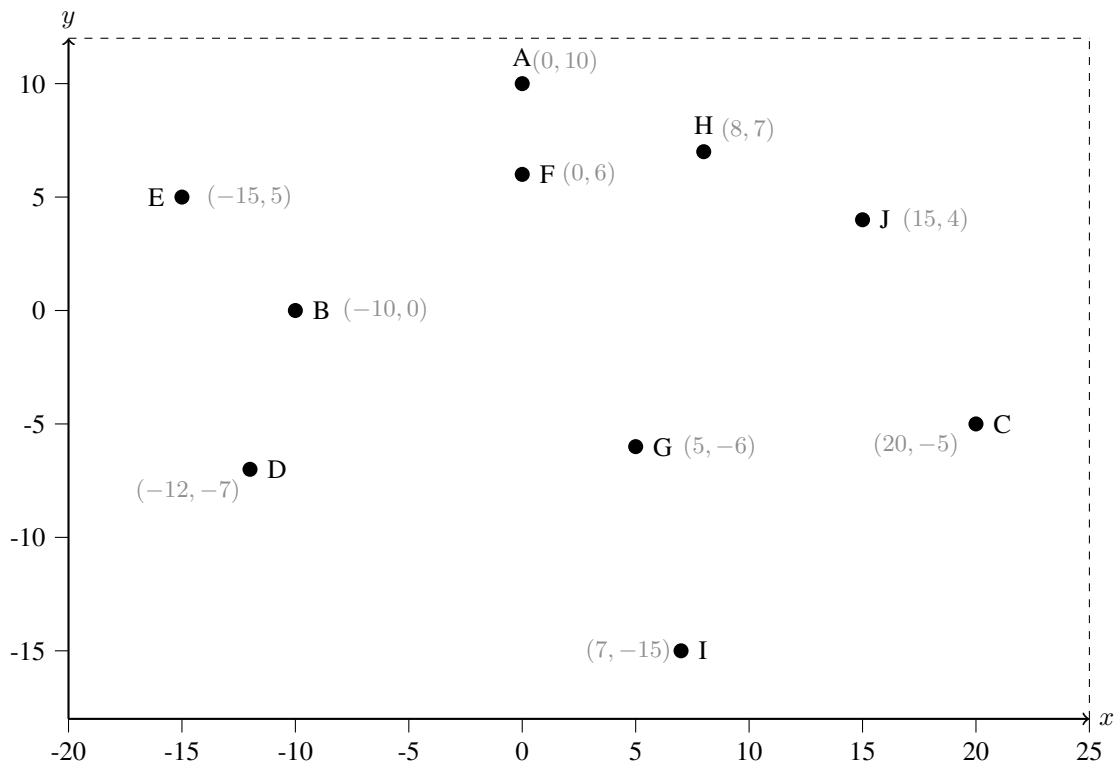
```
1: if points =  $\emptyset$  then
2:   return NIL
3: end if
4: axis  $\leftarrow$  depth %  $k$                                 ▷ Select axis based on current depth
5: points  $\leftarrow$  SORT(points, key = axis)                  ▷ Sort by current axis
6: median_idx  $\leftarrow$  INTEGER(length(points)/2)           ▷ Index of median list item
7: median  $\leftarrow$  KDNode(points[median_idx])
8: left  $\leftarrow$  points[1 ... median_idx - 1]
9: right  $\leftarrow$  points[median_idx + 1 ... length(points)]
10: median.left  $\leftarrow$  BuildKDTree(left, depth + 1,  $k$ )
11: median.right  $\leftarrow$  BuildKDTree(right, depth + 1,  $k$ )
12: return median
```

💡 Conseil

Cet algorithme construit récursivement un arbre KD (KD-Tree) à partir d'une liste de points de dimension k . À chaque appel, il sélectionne d'abord l'axe de séparation en fonction de la profondeur courante (modulo le nombre de dimensions), puis trie les points selon cet axe et choisit le point médian comme racine du nœud courant afin de garantir un arbre équilibré. Les points situés avant ce médian constituent le sous-ensemble gauche et ceux après le médian le sous-ensemble droit. L'algorithme est appelé récursivement ensuite sur chacun de ces deux sous-ensembles pour construire respectivement les sous-arbres gauche et droit. Le processus se poursuit jusqu'à ce qu'il n'y ait plus de points à insérer.

Question 5: (🕒 10 minutes) KD-Tree, un échauffement : Papier

Vous trouverez ci-dessous une liste de points numérotés de 1 à 10. Placez-les dans un KD-Tree et dessinez la séparation de l'espace qui en résulte.



💡 Conseil

La première division se fait de façon verticale. Veillez à bien insérer les points dans l'ordre (point 1, point 2, etc..). Les nœuds se situant au même niveau devraient diviser l'espace selon le même axe.

Question 6: (🕒 15 minutes) KD-Tree : Python

L'objectif de cet exercice est d'écrire une fonction permettant d'ajouter un nœud à un KD-Tree. Les nœuds sont sous la forme $[(x, y), \text{enfant à gauche}, \text{enfant à droite}]$, x et y étant les coordonnées du nœud considéré. Chaque enfant peut être soit un nœud ou une feuille. Complétez le code contenu dans le fichier `Question6.py`.

Voici le pseudo-code permettant d'ajouter un nœud à un KD-Tree :

Algorithm 2 AddKDTree(node, point, cutaxis, k)

```

1: if node = NIL then
2:   new_node  $\leftarrow$  KNode(point)
3:   return new_node
4: end if
5: if point[cutaxis]  $\leq$  node.point[cutaxis] then                                 $\triangleright$  Insert into the left (lesser) subtree
6:   new_cutaxis  $\leftarrow$  (cutaxis + 1) (mod  $k$ )
7:   node.left  $\leftarrow$  AddKDTree(node.left, point, new_cutaxis,  $k$ )
8: else
9:   new_cutaxis  $\leftarrow$  (cutaxis + 1) (mod  $k$ )
10:  node.right  $\leftarrow$  AddKDTree(node.right, point, new_cutaxis,  $k$ )
11: end if
12: return node

```

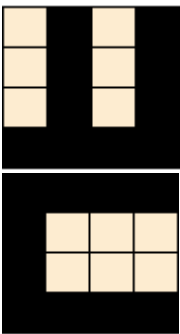
💡 Conseil

Si votre réponse est correcte, votre programme devrait afficher : `[(0, 10), [(-10, 0), None, None], None]`. Pour rappel, en Python, `NIL` correspond à `None`. `KDNode` permet de créer un nouveau nœud, vous pouvez vous inspirer de la structure de `root` définie dans le fichier `Question6.py`.

3 Quad-Tree

Question 7: (🕒 10 minutes) Une mise en train : Papier

Encodez les images ci-dessous dans un Quad-Tree.



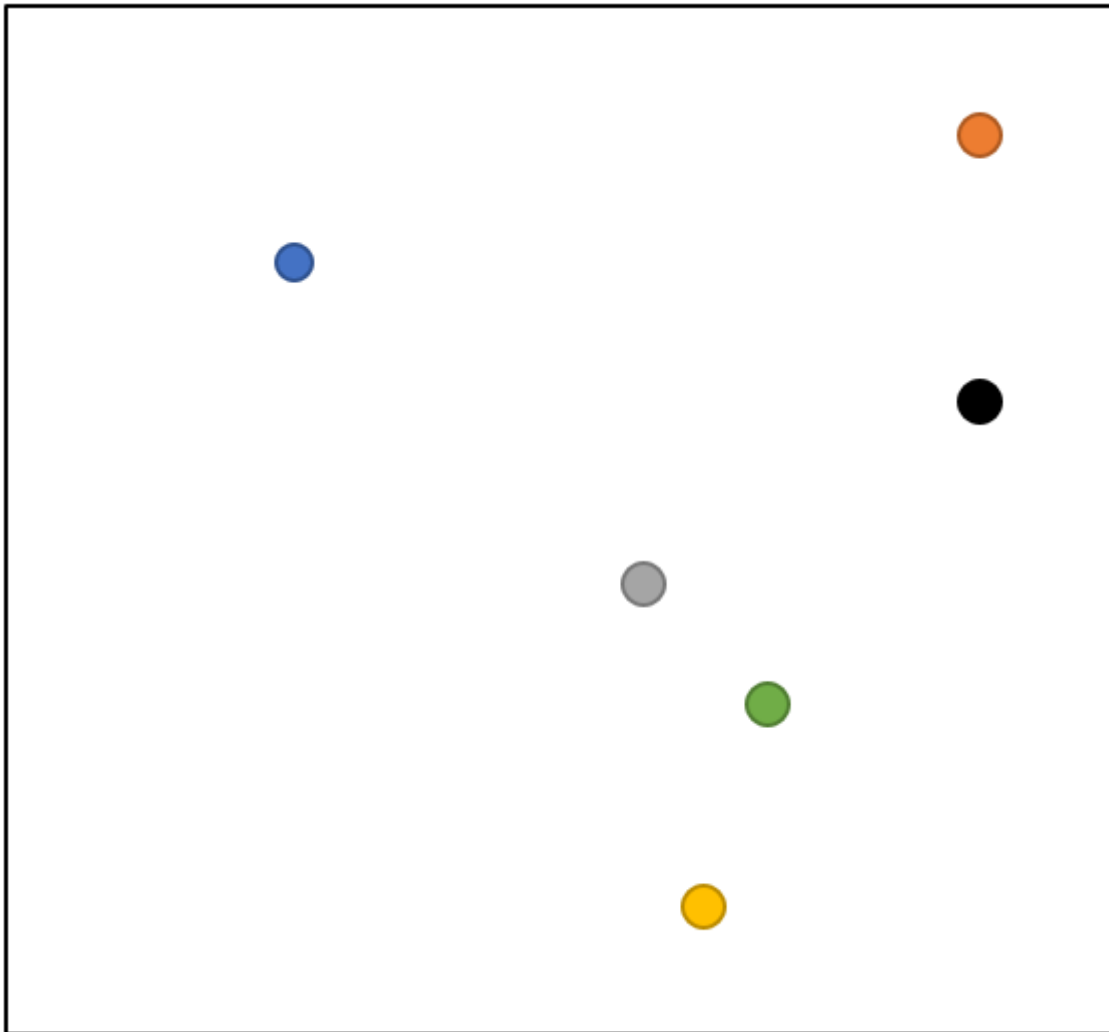
💡 Conseil

Pour réussir cet exercice vous devez diviser chaque nœud en 4 sous-espaces (le plus petit sous-espace étant un carré) de taille égale, et ce autant de fois que nécessaire. La branche la plus à gauche correspond au quadrant NW puis en allant de gauche à droite : NE, SW, SE.

Indice : Votre arbre devrait avoir une profondeur de 2 et disposer de 16 feuilles.

Question 8: (🕒 10 minutes) Une mission capitale : Papier Optionnel

Récemment embauché par la CIA, vous êtes à la recherche d'un individu se cachant dans une des villes suivantes : Bleu, Orange, Noir, Gris, Vert et Jaune. Votre mission, si vous l'acceptez, est de créer un Quad-Tree qui vous permettra de géolocaliser le criminel de façon efficace. Vous trouverez ci-dessous une carte de villes. Créez le Quad-Tree et rétablissez la justice.



 Conseil

Commencez par diviser la carte de la ville de façon adéquate puis construisez le graphe.

Remarque : Les différentes branches de l'arbre n'auront pas toutes la même profondeur.